Software

Kontext

- Wir schreiben hier Software mit vielen Zahlen und Mathematik
- Dies ändert den Kontext in der Softwareentwicklung:
  - **Falsche Ergebnisse** sind Bugs, aber schwerer sichtbar
  - Algorithmen sind oft **komplexer**
  - Schwierigkeiten in der **Numerik**
  - **Grobe Typensysteme** sind oft nicht fein genug
    - numpy.array beschreibt alle möglichen Datentypen
  - **Laufzeiten** sind oft höher; wenige Algorithmen haben quasi-lineare Laufzeit
- Das Softwareengineering sollte daher ergänzt werden
  - Teilweise hilft die Mathematik
- Vieles hier gilt auch für die Algorithmik allgemein.

Zur Programmiersprache

- Ich mache das hier in Python
- Alles was ich mache geht auch in den meisten anderen modernen Programmiersprachen
- Vieles geht in anderen Programmiersprachen besser
- Mich haben bisher vor allem inspiriert:
  - Haskel (alles)
  - Rust (Typsystem)
  - GAP (dispatch, monadisch lernende Objekte)
  - Maple (Termersetzung)
  - Lisp (Homoiconicity: Code als Datenstruktur)

Klassiker, die hier auch gelten

- Versionskontrolle (git)
- Requirementanalyse
- Laufzeitanalyse / Skalierbarkeit
- Sicherheit
- Portierbarkeit / Wiederbenutzbarkeit
- Lifecycle Management
- Bugmanagement

Kommentare im Code

- Let us change our traditional attitude to the construction of programs: Instead of imagining that our main task is to instruct a *computer* what to do, let us concentrate rather on explaining to *human beings* what we want a computer to do.

Donal Knuth, "Literate Programming", *The Computer Journal* **27** (1984), p. 97. (Reprinted in *Literate Programming*, 1992, p. 99.)

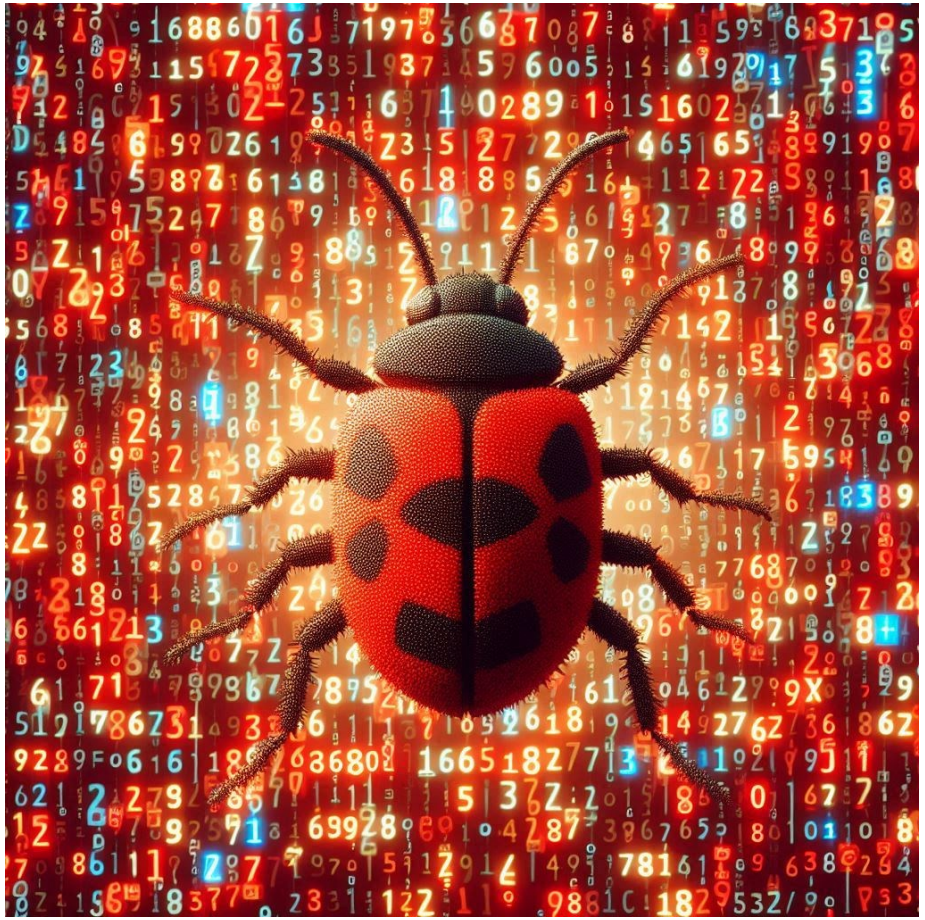
Kommentare im Code

- Kommentare sind wichtig!
  - Fokus auf richtige/wichtige Kommentare
  - Viele (unnütze) Kommentare schaden
- Erkläre *warum* etwas passiert, nicht *was* passiert. Letzteres kann ich im Code direkt lesen.
- Form der Kommentare (Inline, Block, Docstring, Typbeschreibung, …) ist relevant, hängt aber stark von der Programmiersprache und –Umgebung ab.

Kommentare im Code: das sollte kommentiert werden

- Schnittstellen
- Inputs
  - Genaue Beschreibung, was gültige Inputs sind
  - Typen
  - Welche Bedingungen gelten (Sowas ist leider notwendig wegen unflexiblen Typsystemen)
    - Beispiel: „darf nicht `NULL` sein“
    - Beispiel: „>0“
- Outputs
  - Typen
  - wie hängen die Outputs mit den Inputs zusammen
- Falls nicht klar: Begründung der Terminierung
- Falls nicht klar: Begründung der Korrektheit
- Nicht-triviale Schleifeninvarianten oder Rekursionsinvarianten (auch Rekursionshypothese genannt) sollten erklärt werden
  - Gerne auch mit Assertions!
- Verzweigungen sollten „vom hohen Level“ aus erläutert werden (Was bedeutet die Verzweigung? Warum muss ich überhaupt verzweigen?)
- Seiteneffekte sollten sehr klar und ausführlich kommentiert werden
  - Seiteneffekte sind oft ein Antipattern

Debugging von Algorithmen mit Zahlen



„a harmful and bad bug in a lot of digital-looking numbers “ DALL·E 3.

Debugging

- Großes Problem: falsche Ergebnisse
- Fehler sind schwer zu erkennen, wenn Algorithmen nicht verstanden
- Investiere lieber 30 Minuten, den Algorithmus zu verstehen
  - Als 3 Tage zu debuggen
- Wenn Algorithmen schwer nachzurechnen sind, teste einfache Fälle
- Dies bezieht sich auf die Richtigkeit von Berechnungen
  - Aber auch auf die Rechenzeit
- Vergleiche die späteren Kommentare zu „fail fast“

Testing

- Beware of bugs in the above code; I have only proved it correct, not tried it.

Donald Knuth

Testing

Vieles im Testing bleibt gleich

- Alle Methoden sollten mit (Unit-)Tests versehen werden
- Mocking ist manchmal eine gute Idee
  - Komplexität liegt selten in der „Größe“ der Systeme
  - Komplexität liegt eher im Detail der Algorithmen und mathematischen Richtigkeit
  - Daher ist Mocking seltener
- Testing ist komplizierter, da „falsche Ergebnisse“ oft schwerer zu finden sind und keine Exceptions werfen

Code Coverage

```
def multiply(x, y):
• sign = 1
• if x < 0:
    • sign = sign * (-1)
• else:
    • sign = sign * (-1)
• x = math.abs(x)
• if y < 0:
    • sign = sign * (-1)
• else:
    • sign = sign * (-1)
• y = math.abs(y)
• return sign*x*y
```

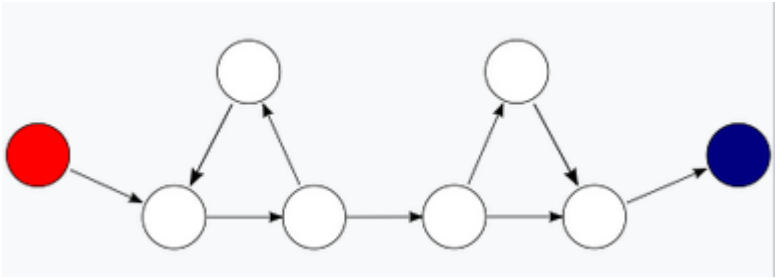
```
# 100% code coverage
import unittest
class
TestMultiply(unittest.TestCase):
• def test_1_neg_neg(self):
    • self.assertEqual(multiply(-2, -2), 4)
• def test_2_pos_pos(self):
    • self.assertEqual(multiply(2, 2), 4)
if __name__ == '__main__':
• unittest.main()
# But:
self.assertEqual(multiply(2, -2),
-4)
```

Cyclomatic Complexity

- 100% Code Coverage reicht nicht.
- Man braucht alle Pfade durch den Code.
  - Oben bräuchte man 2*2=4 Pfade
- Genauer: alle Pfade bis auf Homotopie.
  - Homotopie = „stetige Veränderungen“
  - Bei Pfaden interessiert uns nicht, wie oft eine Schleife durchlaufen wird.
  - (Und ja, es gibt eine sinnvolle und wichtige gemeinsame Verallgemeinerung der Pfade bei zusammenhängenden Mengen und den Pfaden durch den Code.)

Cyclomatic Complexity

- Man kann die Cyclomatic Complexity C wie folgt definieren:
  - Betrachte den Berechenbarkeitsgraph
  - Ergänze eine Kante von jedem **`return`** zum **`begin`**.
  - Dann ist die Cyclomatic Complexity:
    - #Zusammenhangskomponenten-#Knoten+#Kanten
  - Oft also 1-#Knoten+#Kanten
  - Das ist die erste relative Bettizahl des Codes
  - Im linearen Code unterbrochen von einfachen **`if`**s ist es #**`if`**s + 1



https://en.wikipedia.org/wiki/Cyclomatic_complexity#/media/File:Control_flow_graph_of_function_with_loop_and_an_if_statement_without_loop_back.svg, Public Domain

Zum Testen

- Gennerisch braucht man bei Cyclomatic Complexity C höchstens
  - C Tests, um 100% Code Coverage zu erhalten
  - 2^C Tests, um 100% Path Coverage zu erhalten
- Ich würde Tests für 100% Path Coverage anstreben!
  - Das Testen von Code mit vielen Verzweigungen oder Schleifen ist intensiv.
- Überlegen, ob/warum gewisse Pfade ohne Tests nicht auftreten können.
  - Cyclomatic Complexity wird insbesondere geringer, wenn man Code auslagern kann.
  - Das ist für mich der wichtigste Vorteil vom Auslagern von Code.
- Versuche Verzweigungen und Schleifen weitestgehend zu vermeiden.
  - Tricks dazu kommen später.
  - Das ist natürlich nicht absolut möglich, manche Algorithmen brauchen einfach Verzeigungen oder Schleifen.
- Gute Tests nähern einen Beweis des Codes an: alle Fälle werden betrachtet
- Hier wäre viel mehr zu sagen, das überlasse ich anderen Vorlesungen

Warnung: Versteckte Verzweigungen

Es gibt viele versteckte Verzweigungen

- `If a and b:` bedeutet `if a: if b:`
- Diskreten Datenstrukturen verstecken oft Verzweigungen

Gerade in der mathematischen Programmierung gibt es weitere

- `a/b` bedeutet `if not (b==0.0): a/b else: float.nan`
- `log(a)` bedeutet

```
if a>0:
    return log(a)
else:
    if a==0:
        return -float.inf
    else:
        return float.nan
```

- Dies ist beim Testen eine große Herausforderung!

Umgehung von Schleifen

- Monadische Typen wie Listen vermeiden Schleifen
  - ```
    sqr_loop = [] for i in range(1, 11):
        squares_loop.append(i ** 2)
    ```
  - ```
    sqr_list = [i ** 2 for i in range(1, 11)]
    ```
- Weiterer Vorteil: das ist besser parallelisierbar
  - ```
    import concurrent.futures
    with concurrent.futures.ThreadPoolExecutor() as exec:
        sqr_parallel = list(exec.map(lambda n: n ** 2, range(1, 11)))
    ```
  - ```
    import multiprocessing
    with multiprocessing.Pool() as pool:
        sqr_parallel = pool.map(lambda n: n ** 2, range(1, 11))
    ```
- Analoges gilt für Iteratoren, Tupel, Matrizen, …
- Viele Schleifen entsprechen einem Map-Reduce
  - Bspw. Berechnung eines Durchschnitts o.Ä.

Umgehung von Schleifen

- Code kann vereinfacht werden, indem einfacher Code durch Funktoren angereichert wird.
  - Vorteile:
    - Der einfache Code und die Funktoren können unabhängig getestet werden.
    - Der einfache Code kann gelesen werden, ohne die hässlichen Umstände.
    - Hässliche Umstände müssen nur dort betrachtet werden, wo es notwendig ist.
  - Klassisches Beispielklassen von Funktoren:
    - Monaden (Listen, Exceptions, Maybe, State, Reader, Writer, Continuation, …)
    - Comonaden (CoReader, CoWriter, CoState, …)
    - Applicatives (Alle Monaden, Kompositionen von Monaden, ZipList, …)

Umgehung von Schleifen

- Rekursion
  - Schafft kürzere Methoden mit weniger Verzweigungen
  - Manchmal eine Lüge, da eine Rekursion als Ganzes getestet werden muss
  - Früher: Rekursion galt langsam. Moderne Compiler: nicht mehr unbedingt.
    - Rekursion wird vom Compiler in eine Schleife umgewandelt
    - Das klappt besser in stärker typisierten Sprachen

Umgehung von Verzweigungen: Dispatch 1/3: **Dynamic dispatch**

- `-a` ist definiert durch (etwas viele Besserem als):
  - `int.__neg__(self) -> int:`
  - `float.__neg__(self) -> float:`
- Verzweigung automatisch durch den Typ des aufrufenden Objektes
- Wir schreiben also nicht

```
if isinstance(a, int):
    int.__neg__(a)
else if isinstance(a, float):
    float.__neg__(a)
```

- Das Testing ist leicht, jede Methode kann einzeln für sich getestet werden
- Die Auswahl der Methode hängt vom Typen von `a` ab.
- Bei genügend typsicheren Sprachen ist das teilweise zur Compiletime möglich
  - Die Auswahl einer Methode während der Ausführung nennt man **dynamic dispatch**
  - Das ist eine der Haupttechniken objektorientierten Programmierens.

Umgehung von Verzweigungen: Dispatch 2/3: **Single dispatch**

- Mit klassischem dynamic dispatch müssten wir für `a+b` schreiben:
  - ```
    int.__add__(self, b) -> int:
        if isinstance(b,int):
            XXX
        if isinstance(b,float):
            XXX
    ```
- Es gibt zum Glück single dispatch:
  - `int.__add__(self, b: int) -> int:`
  - `int.__add__(self, b: float) -> float:`
  - In Python sind dazu decorators notwendig.
- Das Beispiel macht die Typunterscheidung von `a` durch OOP dynamic dispatch und von `b` durch single dispatch.
- Das Testing ist leicht, jede Methode kann einzeln für sich getestet werden
- Python ist hier komisch, da das dispatch an `self` und `b` hängt,
  also gar nicht an einem „single“-Objekt, sondern an zwei Objekten
    ```

Umgehung von Verzweigungen: Dispatch 2b/3: **Single dispatch**

```
from functools import singledispatchmethod

class MyInt(int):

    @singledispatchmethod
    def __add__(self, other):
        return NotImplemented

    @__add__.register
    def _(self, other: int) -> int:
        print("Adding int to int")
        return int(self) + other

    @__add__.register
    def _(self, other: float) -> float:
        print("Adding int to float")
        return float(self) + other

# Testing
a = MyInt(5)
print(a + 3)
# "Adding int to int" "8"
print(a + 3.0)
# "Adding int to float" "8.0"
```

Umgehung von Verzweigungen: Dispatch 3/3: **Multiple dispatch**

- Warum noch dynamic dispatch für einen Parameter und single dispatch für den anderen?
- `from multipledispatch import dispatch`
- ```
  @dispatch(int, int)
  def add(a, b):
      return a + b
  ```
- ```
  @dispatch(float, float)
  def add(a, b):
  ```
  - `return a + b`
- ```
  @dispatch(int, float)
  def add(a, b):
  ```
  - `return a + b`
- ```
  @dispatch(float, int)
  def add(a, b):
      return a + b
  ```
- Das Testing ist leicht, jede Methode kann einzeln für sich getestet werden.
  - Und man weiß auch viel über die Eingabetypen, was das Testing noch leichter macht.

Umgehung von Verzweigungen: Zusammenfassung Dispatch

- Multiple dispatch klappt auch für beliebig viele Parametern
- Man macht das natürlich nicht nur bei trivialen Operationen
- Typische Anwendungen: Standardwerte zu Beginn setzen
  - Methoden mit weniger Parametern rufen nach und nach Methoden mit mehr Parametern auf
  - Die Methode mit den meisten Parametern macht dann die Rechnung
  - Das ist manchmal flexibler als Standardparameterwerte
  - Besonders falls Standardparameterwerte nicht fix sind, sondern von anderen Parametern abhängen
- Dispatch ist sehr mächtig bei monadischen Typen
- Multiple Dispatch ersetzt das (eingeschränkte) Dynamic Dispatch der Objektorientierheit, was nur an einem Parameter dispatchen kann.

Verzweigungen und Schleifen

- Viele Verwendungen von Verzweigungen und Schleifen sind ein Antipattern, ähnlich wie `goto`.
  - Das ist keine Mehrheitsmeinung, aber meine Meinung.
- Verzweigungen und Schleifen sorgen für viele Bugs, da der Code viele Pfade hat
  - Gleichzeitig wird Testen durch Verzweigungen und Schleifen schwerer
- Viele Verwendungen von Verzweigungen und Schleifen sind vermeidbar
  - Moderne Compiler oder Interpreter sind bei den Alternativen nicht langsamer

Mehrfache Aufrufe

- Um Geschwindigkeit zu sparen werden Methoden oft nur einmal aufgerufen:
  - ```
    x = ExpensiveMethod(variable)
    DoSomething(x)
    SoSomeThingElse()
    DoMore(x)
    ```
- Ich würde wegen Lesbarkeit bevorzugen:
  - ```
    DoSomething(ExpensiveMethod(variable))
    SoSomeThingElse()
    DoMore(ExpensiveMethod(variable))
    ```
- Jetzt sehe ich direkter, was in die Methoden reingeht.
- Die Lesbarkeit wird sogar noch stärker erhöht, wenn die Variable `x` einmal erzeugt wird und in Methoden wiederverwendet wird
- Meine Vermeidung ruft leider `ExpensiveMethod` doppelt auf

Ausweg: Memoization

```
from functools import wraps
def memoize(func):
    cache = {}
    @wraps(func)
    def memoized_func(*args):
        if args in cache:
            return cache[args]
        result = func(*args)
        cache[args] = result
        return result
    return memoized_func
f = lambda x: x ** 2
memoized_f = memoize(lambda x: x ** 2)
print(memoized_f(4)) # Output: 16
print(memoized_f(4)) # Output: 16, retrieved from the cache
```

Bemerkungen zur Memoization

- Die Kosten für die Memoization sind oft vernachlässigbar
- Nachteil: der `cache` speichert Referenzen, welche durch den GarbageCollector nicht freigegeben werden
  - Ausweg 1: Der `cache` kann in der Größe beschränkt werden
  - Ausweg 2: Weak References im `cache`, bspw. durch `WeakValueDictionary`
- Dies ist nur eine tolle Technik aus docs.python.org/3/library/functools.html, wo Memoization in `@functools.cache(user_function)` zu finden ist

Scope/Feature Creep

- Es werden nach- und nach neue Features hinzugefügt
- Code (insbesondere Objekte/Interfaces/Typen) wuchern
  - Großes Problem: mehr Code, mehr Tests, höhere Wartung, mehr Bugs, …
- Mögliche Ursachen:
  1) Schlechte Projektplanung
  2) Entwickler:innen duplizieren vorhandenen Code (große Projekte, wenig Kommunikation, …)
  3) Schlechte Architektur (Objekte/Interfaces/Typen)
- Mögliche Auswege:
  1) Nicht Thema dieser Vorlesung
  2) Bessere Dokumentation, Pair Programming, Code Review, …
  3) Bessere Architekturen
- Zu Punkt 3 möchte ich spezifisch zu mathematischem Code noch einen Kommentar machen

Architektur und Schnittstellen

- Es gibt einen tiefen und formalisierbaren Zusammenhang zwischen (konstruktiver) Mathematik und Informatik (Typentheorie)
  - Und Kategorientheorie von Cartesian Closed Categories
- Die klassische Korrespodenz zwischen Mathematik und Informatik: **Curry–Howard Korrespondenz**
- https://existentialtype.wordpress.com/2011/03/27/the-holy-trinity/



„Howard-Curry-Lambek correspondence, computational trinitarianism“ DALL·E 3.

Architektur und Schnittstellen

- Wörterbuch:
  - Satz <-> Algorithmus
  - Voraussetzungen <-> Eingabe
  - Aussage <-> Ausgabe
  - Beweis(schritte) <-> Rechenschritte
  - Wahrheit <-> Unit Type (hat genau ein Element)
  - Falschheit <-> Empty Type (hat kein Element)
  - Etc.



„Howard-Curry-Lambek correspondence, computational trinitarianism“ DALL·E 3.

Schnittstellen in der Mathematik

- Gewisse Interfaces und Zusammenhänge sind gut durchdacht:
  - Menge, Funktion
  - Mannigfaltigkeit, stetige Funktion, differenzierbare Funktion
  - Vektorraum, lineare Abbildung, Tensorprodukte
  - Gruppe, Operation, Darstellung, Gruppenhomomorphismus
  - Graph, Graphhomomorphismen
  - Etc. etc.
- Solche Schnittstellen sind robust
- Auf diese Schnittstellen bauen viele Operationen auf

Übertragung von Schnittstellen

- Alles, was „**existiert**“, wollen wir **berechnen**.
- Alles, was „**für alle**“ gilt, wollen wir **testen**.
- Definition (Menge): Sei A eine Menge und Ω eine Obermenge. Dann gilt:
$$\forall x \in \Omega: x \in A \vee x \notin A$$
- Beachte: das „Oder“ bedeutet, es existiert eine Entscheidung, ob $x \in A$ oder $x \notin A$.
  - Das heißt, bei einer Menge wollen wir algorithmisch entscheiden, ob ein Element enthalten ist
- Wenn ich also eine Schnittstelle von Mengen festlegen will, dann ist es eine Operation:

```
from abc import ABC, abstractmethod
class ISet(ABC):
    @abstractmethod
    def Contains(self, x: Omega) -> bool:
        pass
```

Datenstrukturen für Mengen

- Von dieser abstrakten Klasse leitet man konkrete Klassen von Mengen ab.
- Diese konkreten Klassen legen den Datentyp der Menge fest:
  - Aufzählende Form: $\{a, b, c, d\}$
  - Typ: $\mathbb{Z}, \mathbb{Q}, \mathbb{N}$
  - Teilmengen durch Entscheidungsfunktion: $\{x \in \mathbb{Q} | x \geq 0\}$
  - Konstruktionen aus Mengen: $\mathbb{Z} \times \mathbb{Z}$, $\mathbb{Z} \cup \{a, b, c\}$

Datenstrukturen für Mengen in der Optimierung

- Mengen X bei uns sind Teilmengen von $\mathbb{R}^d$. Wir schränken also die Definition von Mengen ein. $\mathbb{R}^d$ modellieren wir (sehr vage) durch `np.ndarray`.
- ```
  from abc import ABC, abstractmethod
  import numpy as np
  from multimethod import multimethod
  ```
- ```
  class ISet(ABC):
  ```
- ```
      def __init__(self, ambient_dimension: int):
          self._ambient_dimension = ambient_dimension
  ```
- ```
      @abstractmethod
      def contains(self, point: np.ndarray) -> bool:
           pass
  ```
- ```
      @abstractmethod
      def point(self) -> np.ndarray:
           pass
  ```
- ```
      @property
      def ambient_dimension(self) -> int:
          return self._ambient_dimension
  ```

Beispieldatenstruktur: Affine Räume $\mathbb{R}^d$

```
class AffineSpace(ISet):
    def __init__(self, ambient_dimension: int):
        super().__init__(ambient_dimension=ambient_dimension)
    def contains(self, point: np.ndarray) -> bool:
        return (point.shape == (self.ambient_dimension,))
    def point(self) -> np.ndarray:
        return np.random.randn(self.ambient_dimension)
    @multimethod
    def intersect(self: 'ISet', other: 'AffineSpace') -> 'ISet':
        assert self.ambient_dimension == other.ambient_dimension, "Assume equal
dimensions when intersecting sets.„
        return self
    @multimethod
    def intersect(self: 'AffineSpace', other: 'ISet') -> 'ISet':
        assert self.ambient_dimension == other.ambient_dimension, "Assume equal
dimensions when intersecting sets.„
        return other
    @multimethod
    def cartesian_product(self: 'AffineSpace', other: 'AffineSpace') -> 'AffineSpace':
        return
AffineSpace(ambient_dimension=self.ambient_dimension+other.ambient_dimension)
```

Kommentare

- Woher kommt das `Point`?
  - Für die Optimierung ist das notwendig für die Startpunkte
  - Damit haben wir streng genommen keine Mengen mehr, sondern belebte (inhabited) Mengen (quasi nicht-leere Mengen).
- Lange und aussagekräftige Namen sind eigentlich hilfreich
  - Das macht sich leider auf Folien nicht so gut.

Vergleich zu anderen Designprinzipien

- Andere Designprinzipien
  - Single Responsibility Principle: Jede Schnittstelle hat genau eine Aufgabe (siehe auch Interface Segregation Principle)
  - Microservices: Viele Services kommunizieren über klare Schnittstellen
  - Presentation Layer: Einheitliche Schnittstelle, unabhängig von Datenstruktur darunter
- Diese sind hilfreich im mathematischen Kontext
- Curry-Howard sorgt oft automatisch dafür
- Oft ist a-priori das richtige Abstraktionslevel für Curry-Howard nicht bekannt. Dann lieber auf zu viel Abstraktion verzichten, und später refactorn.

Fail Fast

- Hilfreiches Konzept auf dem Software Engineering: **Fail Fast**
- Wenn irgendwas schief geht, finde es so schnell wie möglich heraus
  - Das hilft beim Debuggen: Fehlermeldungen an richtiger Stelle
  - In Algorithmik umso wichtiger, da Fehler oft gar nicht erkannt werden (Falsche Zahlen)
  - Bei Algorithmen ist es besser eine Fehlermeldung zu haben, als falsche Zahlen
- Falsches Konzept, wenn Programmabstürze schlimmer sind als Fehler



“a software developer moving very fast on a motorcycle while coding and about to crash into a wall“, DALL·E 3.

Fail Fast, Asserts und Verzweigungen

- Asserts in Code einbauen
  - Ein Problem bei Python: kein automatischer Typcheck bei Inputs
    - (Ok, bei Multimethods macht er das, glaube ich.)
    - Hier könnte man Inputtypen checken
    - Aber auch Eigenschaften der Input-Typen
    - Ganz typisch: Inputs checken
  - Wenn man anderen Methoden nicht vertraut, dann auch deren Outputs checken
  - Checks sollten geringe Kosten (Rechenzeit, Speicher, …) haben
- Achtung: Ein gescheiterter Assert könnte auch ein Fehler im Assert sein
- Ich sehe Asserts nicht als weitere Verzweigungen
  - Das sind eher monadische Ergänzungen
- Man beachte, dass obiger Code ohne Verzweigungen auskommt

Automatische Ableitung

- Ziele:
  - Implementiere Ableitungen nur für kurze Codestücke
  - Die Ableitungsregeln setzen alles automatisch zusammen
- Ich habe eine Klasse für differenzierbare Funktionen geschrieben
  - Eher didaktisch, etwas hemdsärmlich
  - Es gibt viele weitere Tricks und deutlich besseren Code, vgl. PyTorch oder JAX

Interface

- ```
  from abc import ABC, abstractmethod
  from Set import ISet, AffineSpace
  import numpy as np
  ```
- ```
  class IDifferentiableFunction(object):
      def __init__(self, name: str, domain: ISet):
          super().__init__()
          self._name = name
          self._domain = domain
  ```
- ```
      @abstractmethod
      def evaluate(self, point: np.ndarray) -> np.ndarray:
          pass
  ```
- ```
      @property
      def domain(self) -> ISet:
          return self._domain
  ```
- ```
      @property
      def name(self) -> str:
          return self._name
  ```
- ```
      @abstractmethod
      def jacobian(self, point: np.ndarray) -> np.ndarray:
          pass
  ```

Eine Standard Datenstruktur

```
from typing import Callable, Union
class DifferentiableFunction(IDifferentiableFunction):
    def __init__(self, name: str, domain: ISet,
                 evaluate: Callable[[np.ndarray], np.ndarray],
                 jacobian: Callable[[np.ndarray], np.ndarray]):
        super().__init__(name=name, domain=domain)
        self._evaluate = evaluate
        self._jacobian = jacobian
    def evaluate(self, point: np.ndarray) -> np.ndarray:
        return self._evaluate(point)
    def jacobian(self, point: np.ndarray) -> np.ndarray:
        return self._jacobian(point)
```

Kommentare zu Interface und Datenstruktur

- Das kommt von Curry-Howard
  - Eine Funktion hat eine Domain und Codomain
    - Mit der Codomain sind wir etwas entspannt und definieren die einfach als $\mathbb{R}^c$
  - Funktionsauswertung
  - Auswertung der Jacobimatrix
  - Name der Funktion (strikt mathematisch unnötig, hilfreich zum debuggen)
- Domain und Name schon in abstrakter Klasse (Interface)
- Eine Standarddatenstruktur, mit λ‘s für die Auswertung
- Auf diese Strukturen können wir flexibel viele Methoden setzen

Methoden im Interface

- ```
  @multimethod
  def __add__(self, other: Union[int, float]) ->
  'IDifferentiableFunction':
      return DifferentiableFunction(
          name="(" + self.name + ") + " + str(other),
          domain=self.domain,
          evaluate=lambda v: self.evaluate(v) + other,
          jacobian=lambda v: self.jacobian(v)
      )
  ```
- ```
  @multimethod
  def __add__(self, other: 'IDifferentiableFunction') ->
  'IDifferentiableFunction':
      return DifferentiableFunction(
          name="(" + self.name + ") + (" + other.name + ")",
          domain=self.domain.intersect(other.domain),
          evaluate=lambda v: self.evaluate(v) + other.evaluate(v),
          jacobian=lambda v: self.jacobian(v) + other.jacobian(v)
      )
  ```

Methoden im Interface

- ```
  def __mul__(self, other: Union[int, float]) -> 'IDifferentiableFunction':
      return DifferentiableFunction(
          name=str(other) + " * (" + self.name + ")",
          domain=self.domain,
          evaluate=lambda v: other * self.evaluate(v),
          jacobian=lambda v: other * self.jacobian(v)
      )
  ```
- ```
  def __pow__(self, power: int) -> 'IDifferentiableFunction':
      return DifferentiableFunction(
          name="(" + self.name + ")^" + str(power),
          domain=self.domain,
          evaluate=lambda v: self.evaluate(v)**power,
          jacobian=lambda v: np.matmul(
              np.array([[power]])*self.evaluate(v)**(power-1), self.jacobian(v))
      )
  ```
- ```
  def __rmul__(self, other: Union[int, float]):
      return self.__mul__(other)
  ```
- ```
  def __sub__(self, other: 'IDifferentiableFunction') -> 'IDifferentiableFunction':
      return self + (-1) * other
  ```

Methoden im Interface

```
def Pairing(self, other: 'IDifferentiableFunction') -> 'IDifferentiableFunction':
    return DifferentiableFunction(
        name="Pair(" + self.name + "," + other.name + ")",
        domain=self.domain.intersect(other.domain),
        evaluate=lambda v: np.concatenate(
            (self.evaluate(v), other.evaluate(v))),
        jacobian=lambda v: np.concatenate(
            (self.jacobian(v), other.jacobian(v)))
    )
```

$$f: A \to B$$
$$g: A \to C$$
$$(f, g): A \to B \times C : a \mapsto (f(a), g(a))$$

Methoden im Interface

```
def CartesianProduct(self, other: 'IDifferentiableFunction') -> 'IDifferentiableFunction':
    proj_domain = AffineSpace(
        self.domain.ambient_dimension+other.domain.ambient_dimension)
    proj1 = DifferentiableFunction.Projection(
        domain=proj_domain, l=range(0, self.domain.ambient_dimension))
    proj2 = DifferentiableFunction.Projection(
        domain=proj_domain, l=range(self.domain.ambient_dimension,
self.domain.ambient_dimension+other.domain.ambient_dimension))
    f1 = DifferentiableFunction.FromComposition(self, proj1)
    f2 = DifferentiableFunction.FromComposition(other, proj2)
    return DifferentiableFunction(
        name="CartesianProduct(" + self.name + "," + other.name + ")",
        domain=self.domain.cartesian_product(other.domain),
        evaluate=lambda v: np.concatenate( (f1.evaluate(v), f2.evaluate(v))),
        jacobian=lambda v: np.concatenate((f1.jacobian(v), f2.jacobian(v)), axis=0)
    )
```

$$f: A \to C$$
$$g: B \to D$$
$$f \times g: A \times B \to C \times D: (a, b) \mapsto (f(a), g(b))$$

Methoden in der Klasse

```
@classmethod
def FromComposition(cls, f: IDifferentiableFunction, g: IDifferentiableFunction) ->
IDifferentiableFunction:
    return cls(
        name="(" + f.name + ") ° (" + g.name + ")",
        domain=g.domain,
        evaluate=lambda v: f.evaluate(g.evaluate(v)),
        jacobian=lambda v: np.matmul(
            f.jacobian(g.evaluate(v)), g.jacobian(v))
    )
```

- Die Kettenregel ist hier sehr klar zu lesen
- In welcher Reihenfolge muss man hier die Komposition bilden?
  - PyTorch oder JAX machen das automatisch richtig.
  - Hier muss man leider nachdenken (und das „Denken“ hier ist NP-hard).

Automatisches Differenzieren

- Das oben war eine Version des automatischen Differenzieren
- Üblicherweise sind andere Verfahren sinnvoller
  - Das ist war mehr, um einen anderen Punkt zu machen
  - Von der Rechenzeit ist obiges Verfahren gar nicht schlecht
  - Es verwendet leider zu viel Speicher
- Viele Machine Learning Bücher beschreiben das recht gut
  - Murphy, Probabilistic ML (Teil 1)
  - Goodfellow et al., Deep Learning

Methoden in der Klasse

```
@classmethod
def Debug(cls, f: IDifferentiableFunction) -> IDifferentiableFunction:
    def modified_evaluate(x):
        result = f.evaluate(x)
        print(f"{f.name} - Input: {x}, Output: {result}")
        return result
    return cls(
        name=f.name,
        domain=f.domain,
        evaluate=modified_evaluate,
        jacobian=f.jacobian
    )
```

- Solche und ähnliche Operationen sind möglich, um das Debuggen zu erleichtern
- (Das alles ist nur eine Auswahl der implementierten Methoden.)

Zusammenfassung für Schnittstellen

- Überlege den Algorithmus in mathematischer Sprache
- Schreibe Schnittstellen in mathematischer Sprache
- Diese Schnittstellen neigen zu großer Einfachheit und Wartbarkeit

Wir werden diese Konzepte weiter verwenden.

Approximation von Ableitungen

- In vielen Fällen ist die Ableitung trotzdem nicht leicht zu berechnen
- Das Zusammensetzen ist gar nicht das Problem
- Einzelschritte können stochastisch sein
  - Dann hilft der Reparametrisierungstrick
- Das Multiplizieren der Jakobimatrizen kann teuer sein
  - In der „richtigen Reihenfolge“ (von hinten nach vorne) sind das nur Matrix-Vektor-Produkte
- Manchmal hilft auch dies nicht. Dann helfen diskrete Approximationen.

Approximation von Ableitungen durch finite Differenzen

- Vorwärtsdifferenzen

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

  - Für Gradienten $d + 1$ Auswertungen (eine in der Mitte, eine in jede Richtung)
- Rückwärtsdifferenzen

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

  - Für Gradienten $d + 1$ Auswertungen (eine in der Mitte, eine in jede Richtung)
- Zentrale Differenzen

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

  - Höhere Genauigkeit als Vorwärts- und Rückwärtsdifferenzen
  - Aber $2d$ Auswertungen für Gradienten
- Bei Gradienten macht man dies Komponentenweise

Fehleranalyse der Ableitung durch finite Differenzen

Hier treten 2 Fehler auf

- Trunkierungsfehler: der Fehler, den ich durch h mache
- Rundungsfehler: der Numerische Fehler, weil Fließkommazahlen schlecht sind
  - Das hängt stark von der Funktion f ab.
- Trunkierungsfehler (unter der Annahme, dass keine Rundungsfehler auftreten):
  - Vorwärtsdifferenzen: $O(h)$
  - Rückwärtsdifferenzen: $O(h)$
  - Zentrale Differenzen: $O(h^2)$
- Für f gutartig: Wähle h^2 grob als Genauigkeit der Fließkommazahl, also h in der multiplikativen Mitte der Genauigkeit der Fließkommazahl
  - Das wäre bei doubles $h \approx 10^{-8}$
  - Wenn f nicht so gutartig, wähle h größer
  - Ich habe als Default mit $h \approx 10^{-5}$ gute Erfahrung gemacht
  - Beim Debugging geht es nicht um höchste Genauigkeit, also ist eine stabile Vorhersage wichtig

Fehleranalyse mit Ableitung durch finite Differenzen

- Teste symbolisch berechnete Ableitung $\nabla_{sym} f$ auf Korrektheit durch finite Differenzen $\nabla_{fd} f$.
- Kriterium: relativer Fehler

$$\frac{||\nabla_{fd} f - \nabla_{sym} f||}{\max(||\nabla_{fd} f||, ||\nabla_{sym} f||)}$$

- Gehe von zentrale Differenzen h geeignet gewählt aus. Sei weiter f hinreichend einfach und differenzierbar (kein neuronales Netz mit 100 Layern und kein ReLU)
  - Wenn die symbolische Ableitung stimmt, dann ist der relative Fehler im numerischen Bereich, sagen wir kleiner als 10^{-6}
  - Wenn der Fehler nur kleiner als 10^{-3} ist, dann würde ich es mir genauer ansehen
  - Wenn der Fehler größer als 10^{-3} ist, dann gehe ich von einem Fehler in $\nabla_{sym} f$ aus
- Beachte:
  - Bei komplizierten Funktionen sollte man besser die Bestandteile testen.
    - Die Funktionen sind doch hoffentlich aus Bestandteilen zusammengesetzt!
      - Separation of Concern
  - Bei nicht-differenzierbarem f kann der relative Fehler auch 2 Größenordnungen größer werden.
  - Es sind natürlich auch sehr oft die finiten Differenzen zu ungenau.

Skalierung

- Fließkommazahlen arbeiten am besten im Bereich $[-1,1]$
- Man sollte durch Skalierung sicherstellen, dass sich alle Zahlen grob in dem Bereich aufhalten
  - Größere Zahlen können zu `float.inf` führen
  - Kleine Zahlen haben weniger Genauigkeit durch wenige Nachkommastellen
- Im maschinellen Lernen
  - skaliert man X-Daten auf $[-1,1]$ oder $N(0,1)$ (ggf. auch transformieren),
  - skaliert man Y-Daten auf $[-1,1]$ oder $N(0,1)$ (ggf. auch transformieren) und
  - initialisiert man die Modelle, so dass $N(0,1)$ (grob) auf $N(0,1)$ abbildet

Skalierung in der Optimierung

- Die zulässige Menge X kann man meistens auch weitläufig in $[-1,1]^d$ reinskalieren
- Die Ausgabewerte der Zielfunktion sind oft vorher nicht bekannt.
  - Dann raten
  - Oder nach ein paar a-prior Auswertungen skalieren
  - Oder nach ein paar Schritten neu skalieren

Skalierung in der Praxis

- $f_{scale} = t_o \circ f \circ t_i$
- Mit $t_0 \colon \mathbb{R} \to \mathbb{R} \colon x \mapsto ax + b$ und $t_i \colon \mathbb{R}^d \to \mathbb{R}^d \colon x \mapsto Dx + c$ für $a, b \in \mathbb{R}$, D eine reelle Diagonalmatrix und $c \in \mathbb{R}^d$.
- Das ist eine Komposition und die kann man auch so handhaben.
  - Wenn die Probleme im Optimierungsalgorithmus liegen, wird man sie oft los
  - Wenn die Probleme in der Auswertung von f liegen, dann leider nicht
- Wenn hochqualitative Optimierungsalgorithmen Probleme machen, dann ist das eine sehr wahrscheinliche Fehlerquelle

Rechenzeiten

- “Experience has shown (see [46], [51]) that most of the **running time** in non-IO-bound programs is **concentrated in about 3 % of the source text**. We often see a short inner loop whose speed governs the overall program speed to a remarkable degree; speeding up the inner loop by 10 % speeds up everything by almost 10 %. And if the inner loop has 10 instructions, a moment's thought will usually cut it to 9 or fewer.”
- “There is no doubt that the grail of efficiency leads to abuse. Programmers waste enormous amounts of time thinking about, or **worrying about, the speed of noncritical parts** of their programs, and these attempts at efficiency actually have a strong **negative impact when debugging and maintenance** are considered. We should **forget about small efficiencies**, say about 97% of the time: premature optimization is the root of all evil.
- Yet we should not pass up our opportunities in that critical 3 %. A good programmer will not be lulled into complacency by such reasoning, he will be wise to **look carefully at the critical code**; but only after that code has been identified. It is often a mistake to make a priori judgments about what parts of a program are really critical, since the universal experience of programmers who have been using **measurement tools** has been that their intuitive guesses fail”

Structured Programming with go to Statements, Donald Knuth, ACM Computing Surveys, 1974

- Meine Erfahrung: die 3% stimmen zwar in algorithmischen Ecken, aber sind sonst viel weniger.

Rechenzeiten

- Kenne Komplexität des *Problems* (P, NP, NC, PSpace, LIN, …)
- Kenne die theoretische Laufzeit des *Algorithmus*, z.B. in O-Notation.
  - Entspricht das der Komplexität des Problems?
- Mache *Profiling* der Implementierung
  - Werden die Codezeilen so oft aufgerufen, wie erwartet?
  - Wird so viel Speicher verwendet, wie erwartet?
  - Skaliert die Laufzeit so wie theoretisch erwartet?
  - Finde approximativ Konstanten für die O-Notation.
  - Finde die (sehr wenigen) kritische Codestellen, die viel Zeit brauchen
  - Finde die (sehr wenigen) Daten, die viel Speicher brauchen

Rechenzeiten

- Rechenzeit
  - Manchmal: Warum braucht die Zeile lange? -> Verbessert Implementierung
    - Sinnvoll in kritischer Stelle
    - Vermeide Verzweigungen
    - Alternative Implementierungen testen
    - Verstehe, was der Compiler tut und optimiere damit
  - Öfter: Warum wird das so oft aufgerufen ? -> Verbessert Algorithmus
    - Schwerer
    - Wichtiger
- Speicherplatz
  - Vermeidung von Duplikation, bspw. wie bei Datenbanknormalisierung
  - Tausche Speicherplatz gegen Rechenzeit, bspw. Generate Data On-Demand (Lazy Evaluation)
  - Compiler-nahe Optimierungen: In-Place, Memory Pooling, Streaming, temporäre Objekte vermeiden, …
- Bei „Verbesserungen“ im Code schauen, ob es wirklich besser wird